B1.P1. PREGUNTA 1.

El programa realiza **N operaciones algorítmicas básicas** que podemos denominar **operaciones SAXPY**. Esto implica que la **complejidad algorítmica** del programa es Θ(**N**), o de **orden lineal** respecto a **N**. Al número de operaciones ejecutadas lo denominamos **OpCount** = **N** (*Operation Count*).

Como la complejidad algorítmica es lineal, aumentar 4 veces el valor de **N** (de un millón a cuatro millones) también aumenta 4 veces la cantidad de operaciones algorítmicas básicas a realizar, es decir, aumenta el **tamaño del problema** 4 veces. Siguiendo con el razonamiento, aumentar 4 veces el número de operaciones básicas supone aumentar (aproximadamente) 4 veces el número de instrucciones máquina ejecutadas (de momento, no sabemos cuántas). Si suponemos que al aumentar el tamaño del problema la productividad del procesador se mantiene constante, es decir, que el valor de **IPC** no varía (relación de instrucciones ejecutadas por ciclo de reloj), y suponemos también que la frecuencia de reloj se mantiene igual (no hay ningún motivo para que cambie), entonces el programa tendrá un tiempo de ejecución 4 veces mayor:

261 us × 4 veces más instrucciones máquina ejecutadas = 1,044 ms.

Se explicará más adelante que la ejecución usando vectores más grandes, que ocupan más espacio en memoria, puede provocar que el rendimiento del procesador se degrade, es decir, puede provocar que se reduzca el valor del **IPC**.

B1.P1. PREGUNTA 2.

Para calcular el número de instrucciones máquina ejecutadas por el programa nos debemos fijar en el código ensamblador, de modo similar que para calcular el número de operaciones básicas del algoritmo nos fijamos en el código en lenguaje C (o Cython). El bucle interno del código ensamblador x86-64 (izquierda) se compone de 7 instrucciones, en las líneas numeradas de la 8 a la 14, incluidas. Reconocemos el bucle gracias a identificar que la instrucción de la línea 14 (**jne – *j*ump *n*ot *e*qual**) es un salto condicional que "salta" a la línea etiquetada con **.L3** (línea 7). Las instrucciones de las líneas 2, 3, 4, 5, 6 y 16 no están dentro del bucle, y se ejecutan solamente una vez, mientras que las instrucciones dentro del bucle (8-14) se ejecutan **N** veces. Por tanto, el nº total de instrucciones máquina ejecutadas (**ICount**) es 7 + 7×**N**. Desde un punto de vista ingenieril, podemos considerar **ICount** = 7×**N** instrucciones, porque la diferencia entre el número exacto y la aproximación es muy pequeña cuando consideramos valores grandes de **N**.

En el caso del código ensamblador RISC-V (derecha), el bucle interno se compone de 8 instrucciones (líneas 4-11). En este ISA (***Instruction Set Architecture***) el salto condicional se representa con el mnemónico **bnez** (***b**ranch if **n**ot **e**qual to **z**ero*), y salta a la línea etiquetada con **.LBB0_1** (línea 3). Ignorando las que están fuera del bucle, podemos aproximar **ICount** como 8×**N** instrucciones. Por tanto:

x86-64 :	ICount (**N** = 1 millón) = 7 millones	ICount (**N** = 4 millones) = 28 millones
RISC-V :	ICount (**N** = 1 millón) = 8 millones	ICount (**N** = 4 millones) = 32 millones

B1.P1. PREGUNTA 3.

Con **N**= 1 millón y para el ISA x86-64 se ejecutan 7**N** = 7 millones de instrucciones máquina.

El **tiempo promedio por instrucción** es 261us / 7 millones de instrucciones = 37,3 ps / instrucción = 0,0373 ns / instrucción

El **tiempo promedio por iteración** del bucle (o por operación SAXPY) es 261 us / 1 millón de iteraciones = 0.261 ns por iteración (o operación SAXPY)

1 s (second)	= 1000 milliseconds (ms)
1 ms	= 1000 microseconds (µs or us)
1 us	= 1000 nanoseconds (ns)
1 ns	= 1000 picoseconds (ps)

B1.P1. PREGUNTA 4.

Si el programa completo (10 mil repeticiones de la ejecución de la función **F**) se ejecuta en 2,61 segundos, y esto corresponde con un tiempo medido en ciclos de reloj de 11,43 giga ciclos, entonces se puede estimar la frecuencia de reloj como:

11,43 Giga ciclos / 2,61 segundos = 4,379 GHz ≈ **4,38 GHz**

Las 7 instrucciones del bucle interno se ejecutan 10.000 × **N** = diez mil millones de veces, lo que da un total de 70 Giga instrucciones. Si al recuento total de instrucciones que nos proporciona **perf**, que es 70,196 giga, le restamos el recuento de las instrucciones ejecutadas dentro del bucle, que son 70 giga, obtenemos **196 millones de instrucciones** ejecutadas que no corresponden al bucle interno. Este valor supone un 0,196 / 70 = 0,0028 -> 0,3% del total de las instrucciones que se ejecutan en el bucle interno, así que, desde un punto de vista ingenieril, podemos ignorar el efecto de estas instrucciones y asumir que los resultados que obtengamos en nuestros cálculos serán bastante aproximados.

Finalmente, dividiendo el total de ciclos de reloj que toma la ejecución del programa, 11,43 giga, entre el total de iteraciones del

bucle, 10.000 × **N** = diez mil millones = 10 giga, obtenemos un promedio de 11,43 / 10 = **1,14 ciclos por iteración**.

La relación entre el número de operaciones básicas (SAXPY) ejecutadas por el programa, 10 giga, y el número de instrucciones máquina ejecutadas, 70,196 giga, la denominamos **eficiencia de la codificación del programa**. La relación la podemos calcular como el número de instrucciones máquina por operación básica, es decir **InstPerOp** = 7,02 instrucciones / operación. La ventaja de esta **métrica de rendimiento** es que se relaciona directamente con el código ensamblador: por cada operación SAXPY hay que ejecutar siete instrucciones, que son las que encontramos dentro del bucle, y 0,02 instrucciones de promedio de las instrucciones fuera del bucle. Es decir, sin tener que visualizar el código ensamblador del bucle interno del programa, y solamente usando los resultados de la utilidad **perf**, podemos estimar el número de instrucciones máquina que se ejecutan en el bucle interno, y valorar si este valor tiene sentido, o es superior a lo esperado; y esto nos puede guiar para analizar el programa con más profundidad y, tal vez, nos puede inducir a transformarlo para mejorar su rendimiento.

La desventaja de la métrica **InstPerOp** es que es de tipo "*cuanto menor mejor*", que no es tan intuitivo como las métricas de tipo "*cuanto mayor mejor*". Podemos expresar una métrica de rendimiento alternativa que sea la inversa: Operaciones por Instrucción (**OpPerInst**, *Operations per Instruction*), calculada como 10 giga iteraciones / 70,196 giga instrucciones = 0,142. Como en general se necesitan varias instrucciones máquina para codificar una operación básica del algoritmo, podemos calcular las operaciones básicas codificadas cada mil instrucciones máquina: **OpPerInst** = 1000 × 0,142 = 142 operaciones SAXPY cada mil instrucciones ejecutadas.

B1.P1. PREGUNTA 5.

Una métrica muy útil para **valorar** el rendimiento de la ejecución de un programa es el **IPC** (*Instructions Per Cycle*), que es el promedio de instrucciones máquina ejecutadas por unidad de tiempo, medido en ciclos de reloj. A esta métrica se le puede llamar **productividad o desempeño del procesador**. Se calcula dividiendo el nº total de instrucciones ejecutadas entre el nº total de ciclos de reloj que se demora la ejecución. En el ejemplo, **IPC** = 70,196 giga instrucciones / 11,427 giga ciclos = **6,143**. Es muy importante fijarse en que el procesador es capaz de ejecutar un promedio de más de 6 instrucciones durante cada ciclo de reloj. Esto nos indica que el procesador puede estar ejecutando múltiples instrucciones en paralelo (también se puede decir: de forma simultánea o concurrente).

Si tal como habíamos deducido, un núcleo de cómputo del procesador es capaz de ejecutar un promedio de 6 instrucciones cada ciclo de reloj, entonces un **IPC** de 6,142 nos indica que estamos utilizando un 102,4% de la capacidad de cómputo máxima del procesador. Expresado de manera formal, **Eficiencia** = $IPC_{obtenido}$ / $IPC_{máximo}$ = 6,142 / 6 = 1,024 → **102,4%**. Evidentemente, esto no tiene mucho sentido. La explicación de esta "magia" es que en tiempo de ejecución el front-end del procesador está fusionando instrucciones, de modo que las 7 instrucciones del bucle que vemos en el programa ensamblador, y que están almacenadas en la memoria del procesador, en realidad se están convirtiendo en menos de 7 micro-operaciones, probablemente en 6 o 5. Como conclusión, es difícil entender cómo se están ejecutando un programa si tanto el compilador como el procesador realizan transformaciones en él. Pero durante este curso vamos a explicar las ideas más importantes, y os van a servir para diseñar programas mucho más eficientes.

Una métrica de rendimiento alternativa para medir el *throughput* del procesador es el **CPI** (*Cycles Per Instruction*), que corresponde con el inverso del **IPC**, y que se calcula como CPI = 11,43 giga ciclos / 70,196 giga instrucciones = 1 / 6,143 = 0,163. Esta métrica cumple la regla de "*cuanto menor mejor*", lo cual no es tan intuitivo, pero además nos puede inducir a interpretar que la **latencia** media de ejecución de las instrucciones es de 0,163 ciclos. Esto NO ES CIERTO en los procesadores actuales, porque la ejecución de una instrucción requiere al menos 1 ciclo de reloj para ejecutarse, y en muchos casos se requieren varios ciclos; lo que ocurre es que los procesadores son capaces de ejecutar más de una instrucción de forma simultánea (en paralelo).

B1.P1. PREGUNTA 6.

Usaremos la ecuación del rendimiento siguiente: se calcularán los tres factores por separado y se multiplicarán entre sí.

$$\frac{\text{Basic Operations}}{\text{Second}} = \frac{\text{Basic Operations}}{\text{Instruction}} \; \frac{\text{Instructions}}{\text{Clock Cycle}} \; \frac{\text{Clock Cycles}}{\text{Second}}$$

El primer factor es **OpPerInstr**: cada operación SAXPY se codifica en el repertorio RISC-V con 8 instrucciones, **OpPerInstr** = 1 / 8 = 0,125. Es una aproximación, porque asumimos que la cantidad de instrucciones ejecutadas fuera del bucle interno del programa son relativamente pocas, y las ignoramos.

El segundo factor es el **IPC**: si es un 60% del máximo posible, que es 4, esto implica que el **IPC** es 4 × 0,60 = 2,4.

El tercer factor es la **frecuencia de reloj**: nos la dan en el enunciado, y es 1,6 GHz = 1,6 giga ciclos / segundo

Por tanto, se ejecutan 0,125 × 2,4 × 1,6 GHz = 0,48 giga operaciones básicas SAXPY por segundo

B1.P10

a.1) Mejora debida a la modificación de la función A

T_{Anew} = 400s / 2 = 200s. SpeedUp $_A$ = (1000s) / (200s+350s+250s) = **1,25×**

Otra manera de hacerlo, aplicando la fórmula de la Ley de Amdhal:

A es un 40% (0,4) del tiempo total del programa. SpeedUp $_A$ = 1 / (1-0,4 + 0,4/2) = **1,25×**

a.2) Mejora debida a la modificación de la función B

T_{Bnew} = 350s / 1,75 = 200s. SpeedUp $_B$ = (1000s) / (400s+200s+250s) = **1,18×**

B es un 35% (0,35) del tiempo total del programa. SpeedUp $_B$ = 1 / (1-0,35 + 0,35/1,75) = **1,18×**

La modificación de la función A proporciona un mayor *speedup* global (1,25×)

b) SpeedUp $_{AB}$ = (1000s) / (200s+200s+250s) = **1,54×** **o** SpeedUp $_{AB}$ = 1 / (1-0,75 + 0,40/2 + 0,35/1,75) = **1,54×**
A y B son un 75% (0,75) del tiempo total

Aplicando los dos cambios se obtiene un speedup de **1,54×**

c) Tiempo de nueva función C: t_{Cnew}

SpeedUp $_{ABC}$ = 1000s / (200s + 200s + t_{Cnew}) = 2
Despejando t_{Cnew}, se obtiene t_{Cnew} = 100 s. Por lo tanto, SpeedUp $_C$ = 250 s / 100 s = **2,5×**

Alternativamente, se puede calcular usando la Ley de Amdahl

SpeedUp $_{ABC}$ = 2; 1 / (0,4/2 + 0,35/1,75 + 0.25/SpeedUp $_C$) = 2

(0,4 + 0,25/SpeedUp $_C$)$^{-1}$ = 2 ; (1-0,8) SpeedUp $_C$ = 0,5 ; SpeedUp $_C$ = **2,5×**

B1.P11

(a) El bucle interno consta de 7 instrucciones (líneas 11-17), y se ejecuta N^2 veces (complejidad computacional cuadrática debida a que se usan dos bucle anidados, cada uno realizando N iteraciones)
ICount = 7 × N^2 = 7 × $(50K)^2$ = 17,5 Giga instrucciones; 4 segundos × 3 GHz = 12 G ciclos;
IPC = 17,5 G instr. / 12 G ciclos = 1,4583

(b) ICount = 7 × $(80K)^2$ = 44,8 Giga; Tiempo = 44,8 G instr. × (1 ciclo / (1,4583/2) instr.) × (1 / 3 GHz) = 20,48 segundos

(c) El bucle del programa tiene 7 instrucciones, que corresponden con las líneas de la 11 a la 17, incluida, y, por tanto, no hay mejora en la codificación de instrucciones. La frecuencia de reloj empeora 3 GHz / 1,2 GHz = 2,5 x. El IPC mejora 1,8x, y si multiplicamos las mejoras obtenemos: 1,8 × (1/2,5) = 0,72 x, que en realidad no es una mejora, sino que supone reducir el rendimiento un 28%.

Otra forma de resolver el último apartado es calcular el tiempo de ejecución en el procesador RISC-V:

El IPC en el nuevo procesador es 1,4583 × 1,8 = 2,625.

El tiempo de ciclo del reloj, con una frecuencia de 1,2 GHz es de 1 / 1,2 = 0,833 ns.

Tiempo = 7 × $(50K)^2$ instr. × (1 / 2,625 instr. /ciclo) × 0,833 ns / ciclo = 17,5 Giga instr. × 0,31745 ns / instr = 5,556 s.

La mejora es 4 / 5,556 = 0,72x

B2.P1

1. Escribid el código en lenguaje C equivalente al código ensamblador.

```c
double f (double X[], int N)
{
  int i=0;
  double sum=0.0;
  for (; i<N; i+=2)
  {
    sum += X[i];
    sum += X[i+1];
  }
  return sum;
}
```

```
11  .L3:
12          addsd   (%rax), %xmm0
13          addsd   8(%rax), %xmm0
14          addq    $16, %rax
15          cmpq    %rdx, %rax
16          jne     .L3
```

Abstract ASM description

```
%xmm0: sum    %rax: X
    %rdx: ptrEnd=X+N

sum = sum + X[0]
sum = sum + X[1]
X   = X + 2
c   = X != ptrEnd
if c branch to beginning
```

Ahora es el registro **%rax** el que se usa como apuntador al array **X**. La instrucción de la línea 13 usa un modo de direccionamiento que incluye la constante 8 para calcular la dirección de memoria de la que leer, y que se denomina **desplazamiento**. El registro **%rax** se incrementa de 16 en 16, porque en cada iteración del bucle se procesan dos datos de tipo **double**, que ocupan 8+8=16 bytes, y el programa ha de "saltar" en el array de 2 en 2

2. Calculad la relación de mejora que se produce en el número total de instrucciones ejecutadas (ICount).

El programa original ejecuta aproximadamente ICount = 4×**N** instrucciones (suponemos que **N** es un valor grande, así que podemos ignorar las instrucciones fuera del bucle).

El programa "desenrollado" ejecuta aproximadamente ICount = 5×**N**/2 = 2,5×**N** instrucciones. El bucle interno tiene 5 instrucciones, pero el bucle itera la mitad de las veces, o, dicho de otra manera, las 5 instrucciones del cuerpo del bucle se ejecutan la mitad de veces que en la versión original, porque los elementos del array **X** se procesan de 2 en 2.

La relación de mejora en la eficiencia de codificación del programa es (4×**N**) / (2,5×**N**) = **1,6x**.
Esto puede suponer una mejora potencial de 1,6x en el tiempo de ejecución, pero más adelante veremos que esto no es necesariamente cierto, y que la mejora del tiempo puede ser nula (1x), mayor a 1,6x o incluso negativa (speedup < 1).

3. Identificad el posible problema funcional y proponed una modificación del código para arreglar este problema

Cuando **N** es un valor impar, el nuevo programa leerá el elemento **X[N]**, que está fuera del array y acumulará el valor en la variable **sum**, produciendo un resultado imprevisible, probablemente incorrecto. El lenguaje C, en contraste con otros lenguajes, como Python, no genera ningún tipo de control para comprobar si los accesos al vector se hacen fuera de rango (*out of bounds* o *out of range*), tal como se ve en el código ensamblador. El lenguaje C prima la generación de código rápido frente a la protección al programador, que hace el lenguaje Python. La solución para que el programa sea siempre funcionalmente correcto es añadir código previo al bucle, lo que se denomina un **prólogo**, para comprobar este caso concreto, tal como se muestra a la derecha.

```c
double f (double X[], int N)
{
  int i=0;
  double sum=0.0;
  if (N & 1 == 1) {
    sum = X[0];
    i = 1;
  }
  for (; i<N; i+=2)
  {
    sum += X[i];
    sum += X[i+1];
  }
  return sum;
}
```

La operación "**N** & 1" realiza una operación AND bit a bit del valor **N** con el valor 1. El resultado de esta operación será 0 cuando el bit menos significativo del valor **N** sea 0, y el resultado será 1 cuando el bit menos significativo del valor **N** sea 1. Es decir, el resultado es 1 si **N** es impar. Este "truco" se basa en entender lo que significa la codificación binaria de los números y lo que hacen las operaciones bit-a-bit, como AND, OR y XOR (en lenguaje C se usan los operadores "&", "|" y "^"). El uso de estos "trucos" puede ser una forma efectiva de optimizar ciertas operaciones con números enteros para que se ejecuten más rápidamente. En este ejemplo, la optimización no tiene mucho efecto porque está en una parte del código que no es el "*hot spot*".

4. Explicad las diferencias que se producen en el código ensamblador y proponed una hipótesis para explicarlas.

Se ha **intercambiado** el orden de las instrucciones de las líneas 13 y 14, pero para que el código reordenado sea equivalente al anterior, es necesario que en el cálculo de la dirección de memoria que se hace en la instrucción **addsd** de la línea 14 se "corrija" la suma de 16 unidades que se le ha hecho al registro **%rax** "antes de tiempo". La corrección implica que, en lugar de usar un desplazamiento de 8 Bytes, se usa un desplazamiento de -8 Bytes, es decir, de 16 Bytes menos.

```
11  .L3:
12          addsd   (%rax), %xmm0
13          addq    $16, %rax
14          addsd   -8(%rax), %xmm0
15          cmpq    %rdx, %rax
16          jne     .L3
```

Hipótesis: veremos más adelante en el curso que el orden en que las instrucciones se procesan en la CPU afecta al tiempo de ejecución. El compilador modifica el orden de ejecución porque entiende que de esta manera el programa se puede ejecutar más rápido. En realidad, en los procesadores actuales, el efecto de esta transformación en el rendimiento es mínimo o nulo.

Universitat Autònoma de Barcelona

B2.P2

1. Escribid el código C equivalente y el papel de los registros: **%rax**, **%rdx**, **%rdi** y **%rsi**.

%rax = k

El registro **%rax** se usa como contador del bucle **for** (tipo **int**) pero también como índice para formar una dirección de memoria. La instrucción **testl**, de la línea 16, usa solamente los 32 bits menos significativos del registro **%rax**, que se representan como **%eax**. En cambio, en las instrucciones de las líneas 11, 12 y 15, el registro **%rax** se usa con sus 64 bits completos. El compilador es responsable de que este uso mixto del registro cumpla con la funcionalidad esperada del lenguaje C. Nuestro interés está en el rendimiento de la ejecución, que no varía si se usan datos enteros de 32 bits o de 64 bits.

Abstract ASM description

```
tmp  = F[k]            (o X[i+k])
tmp  = tmp × X[i+k]    (o  F[k] )
tmp  = tmp + O[i]
O[i] = tmp
k    = k - 1      // index addition
c    = k != 0
if c branch to beginning
```

%rsi = **O**+i **%rdx** y **%rdi** = **F** y **X**+i

Deducimos que **%rsi** contiene la dirección al vector **O** más el índice **i**, porque se usa como dirección para leer y **para escribir** en memoria (instrucciones 13 y 14). Para los otros dos registros, no podemos saber cuál "apunta" a qué vector sin mirar el código de fuera del bucle interno, porque tanto **F**[] como **X**[i] se acceden para leer sendos valores que luego solamente se usan para multiplicarse entre sí, y la operación de multiplicar es conmutativa (da igual A*B que B*A).

2. Proponed un cambio en el programa fuente.

La optimización consiste en substituir temporalmente la posición de memoria **O**[i] por un registro, **%xmm1**: en el código fuente es equivalente a substituir la posición de memoria **O**[i] por la variable local **Oi**, tal como se muestra a continuación. De este modo, movemos la lectura (LOAD) y la escritura (STORE) al array **O**[], que está en memoria, fuera del bucle interno (***code motion***). El resultado es que durante la ejecución del programa se evitan dos accesos a memoria, una lectura y una escritura, en cada iteración del bucle más interno.

```
void F ( float X[], float O[], float F[], int N, int K ) {
  for (int i=0; i<N; i++) {
    float Oi= O[i];
    for (int k=K-1; k>=0; k--)
      Oi += F[k]*X[i+k];
    O[i] = Oi;
} }
```

```
void F ( float X[], float O[], float F[], int N
{
  for (int i=0; i<N; i++)
  {
    float Oi= O[i];
    for (int k=K-1; k>=0; k--)
    {
      Oi += F[k]*X[i+k];
    }
    O[i] = Oi;
  }
}
```

x86-64 gcc 11.2 -O1

A Output... Filter... Libraries + Add new.

```
13  .L4:
14      movss   (%rdx,%rax,4), %xmm0
15      mulss   (%rdi,%rax,4), %xmm0
16      addss   %xmm0, %xmm1
17      subq    $1, %rax
18      testl   %eax, %eax
19      jns     .L4
```

Abstract ASM description

```
Register %xmm0 is temp
Register %xmm1 is Oi
Register %rax is i
Registers %rdi & %rdx hold addresses X+i & F

temp = F[k]                (o X[i+k] )
temp = temp * X[i+k]       (o  F[k]  )
Oi   = Oi + temp
i    = i - 1
c    = i == 0
if !c branch to beginning
```

Universitat Autònoma de Barcelona

B2.P5

1. Calcula el número promedio de instrucciones máquina ejecutadas en cada iteración del bucle del código **x86-64**.

El cuerpo del bucle mostrado a la derecha se compone de 4 grupos de instrucciones o **bloques básicos**, A, B, C y D, cada uno compuesto de 2, 6, 4 y 3 instrucciones, respectivamente.

Pueden ocurrir dos casos diferentes en cada iteración del bucle:

CASO 1: se ejecutan los bloques C, A y B: 4+2+6 = 12 instrucciones

CASO 2: se ejecutan los bloques C, D y B: 4+3+6 = 13 instrucciones.

El porcentaje de cada caso depende de los datos concretos que tienen los vectores de entrada a la función. En el "mejor" de los casos siempre se da el caso 1, y entonces se ejecutan 12 instrucciones por iteración del bucle. En el "peor" de los casos siempre se da el caso 2, y entonces se ejecutan 13 instrucciones por iteración del bucle. En todos los casos ignoramos las instrucciones que se ejecutan fuera del bucle, que representan un valor muy pequeño en comparación con el número de veces que se ejecutan las instrucciones que están dentro del bucle.

Instrucciones / Iteración = entre **12 y 13**

```
x86-64 gcc 12.2        -O3 -fopt-info
A  Output...  Filter...  Libraries  + Add new...
 9  .L14:
10          subsd   %xmm2, %xmm1        } A
11          movapd  %xmm1, %xmm0
12  .L5:
13          movsd   %xmm0, (%rdx,%rax)  } B
14          maxsd   %xmm3, %xmm0
15          addq    $8, %rax
16          movapd  %xmm0, %xmm3
17          cmpq    %rax, %rcx
18          je      .L1
19  .L7:
20          movsd   (%rdi,%rax), %xmm2  } C
21          movsd   (%rsi,%rax), %xmm1
22          comisd  %xmm2, %xmm1
23          ja      .L14
24          subsd   %xmm1, %xmm2        } D
25          movapd  %xmm2, %xmm0
26          jmp     .L5
```

2. Haz el mismo cálculo para el código **rv64gc**.

Ahora el cuerpo interno del bucle se compone de 6 grupos de instrucciones o **bloques básicos** (A, B, C, D, E y F, cada uno compuesto de 4, 1, 6, 1, 1 y 2 instrucciones, respectivamente).

Pueden ocurrir dos casos diferentes en iteración del bucle:

CASO 1: se ejecutan A, B, C, D, E: 4+1+6+1+1 = 13 instrucciones

CASO 2: se ejecutan A, B, C, E: 4+1+6+1 = 12 instrucciones

CASO 3: se ejecutan A, F, C, D, E: 4+2+6+1+1 = 14 instrucciones

CASO 4: se ejecutan A, F, C, E: 4+2+6+1 = 13 instrucciones

De nuevo, el porcentaje de cada caso depende de los datos concretos que tienen los vectores de entrada a la función El "mejor" caso es el caso 2 y el "peor" es el caso 3:

Instrucciones / Iteración = entre **12 y 14**

```
 6  .L7:
 7          fld     fa5,0(a0)           } A
 8          fld     fa4,0(a1)
 9          flt.d   a5,fa5,fa4
10          beq     a5,zero,.L12
11          fsub.d  fa5,fa4,fa5         } B
12  .L5:
13          flt.d   a5,fa0,fa5          } C
14          fsd     fa5,0(a2)
15          addi    a0,a0,8
16          addi    a1,a1,8
17          addi    a2,a2,8
18          beq     a5,zero,.L6
19          fmv.d   fa0,fa5             } D
20  .L6:
21          bne     a0,a4,.L7           } E
22          ret
23  .L12:
24          fsub.d  fa5,fa5,fa4         } F
25          j       .L5
```

3. Explica ambas transformaciones y el valor de desplazamiento = 8

```
for (int i=0; i<N; i+=2) {
    if (A[i] < B[i])
      C[i] = B[i]-A[i];
    else
      C[i] = A[i]-B[i];
    if (Max < C[i])
      Max = C[i];
    if (A[i+1] < B[i+1])
      C[i+1] = B[i+1]-A[i+1];
    else
      C[i+1] = A[i+1]-B[i+1];
    if (Max < C[i+1])
      Max = C[i+1];
}
```

(1) ***Register reuse***: la instrucción 29 escribe el contenido de %xmm1 en memoria, en el elemento **C**[i], y se puede reutilizar el contenido del registro %xmm1 para compararlo con el contenido de la variable **Max** (registro %xmm0) con la instrucción 30, sin tener que volver a leerlo de memoria.

(2) ***Loop unroll***: cada iteración del bucle del programa ensamblador hace el trabajo de **dos iteraciones** del bucle del programa fuente en lenguaje C. El resultado es equivalente al caso en que la programadora hubiera modificado el bucle como se muestra a la derecha (suponemos que el valor de N es siempre par). En el código ensamblador existe un código especial, que no se muestra, que gestiona el caso en que el valor de N sea impar.

La razón de que el contador del bucle (*%rax*) se incremente de 2 en 2 es que cada iteración del bucle procesa 2 elementos de cada vector (de **A**[] y de **B**[]) y genera 2 resultados (que se escriben en el vector **C**[]). El segundo elemento de cada vector, **A**[i+1], **B**[i+1] o **C**[i+1], debe incluir el desplazamiento 8, porque el valor 8 corresponde con el tamaño de un elemento del vector: sizeof(double) = 8 Bytes.

Direccionamiento con escalado 8x al elemento **A**[i]
`movsd (%rdi,%rax,8), %xmm1` →

Direccionamiento con escalado 8x al elemento **A**[i+1]
`movsd 8(%rdi,%rax,8), %xmm1`

4. Calcula la relación de mejora (*speedup*) respecto al número total de instrucciones máquina ejecutadas.

El cuerpo interno del bucle del nuevo programa ASM se compone de un único **bloque básico**, que tiene 27 instrucciones. Pero cada iteración del bucle del programa ASM realiza el equivalente a dos iteraciones del bucle del programa original. Así, hemos de considerar un promedio de 27/2 = 13,5 instrucciones / iteración. Comparando con la primera versión, que ejecutaba entre 12 y 13 instrucciones por iteración, esta nueva versión es peor.

Peor caso: speedup = 12 / 13,5 = 0,89 (speed down ➔ reducción aproximada del 11%)

Mejor caso: speedup = 13 / 13,5 = 0,96 (speed down ➔ reducción aproximada del 4%)

5. ¿Por qué no aparece la llamada a la función **fabs**() en el código ensamblador?

La optimización se denomina ***function inlining***, y se activa con la opción –O3. También se puede activar o desactivar por separado con las opciones **–finline** y **–fno-inline**.

El compilador substituye el código de la función **fabs**(), declarado en el fichero **math.h**, dentro del bucle interno de la función **AbsDiff2**(). Hay tres ventajas de esta transformación: (1) se evita ejecutar las instrucciones ***call*** y ***ret***, de llamada a función y retorno de función; (2) no es necesario mover datos de unos registros a otros para pasarlos como parámetros a la función y para copiar el resultado de la función al registro que sea necesario; y (3) el código ASM de la función se puede combinar con el código ASM del bucle para generar un código más rápido, por ejemplo, reduciendo la cantidad total de instrucciones ejecutadas.

6. ¿Cómo evitar la instrucción de la línea 16 de la función **AbsDiff2**, **movapd**?

La instrucción **movapd** sirve para mover el valor que hay en el registro **%xmm0** al registro **%xmm1**, y el valor que hay en el registro **%xmm0** se obtiene de una instrucción previa que calcula el máximo entre los valores que hay en **%xmm1** y en **%xmm0** (**`maxsd %xmm1, %xmm0`**). La forma de evitar la instrucción es cambiar el orden de los operandos de la instrucción que calcula el máximo:

```
maxsd  %xmm1, %xmm0        →        maxsd %xmm0, %xmm1
movapd %xmm0, %xmm1
```

El cambio lo podemos hacer porque la operación de calcular el máximo es conmutativa: el resultado es el mismo independientemente del orden de los factores. Es "extraño" que el compilador no "sepa" hacer este cambio, pero hemos de recordar que el compilador es un programa muy complejo, y que la tarea de optimizar también es extremadamente compleja.

7. ¿Qué hace la instrucción de la línea 15 del código **x86-64**, **addq**, y por qué se usa el valor 8?

Como ahora no se usa direccionamiento a memoria con el escalado **8x**, el índice que contiene el registro *%rax* se ha de incrementar de 8 en 8, en lugar de hacerlo de 1 en 1, que es como se debería hacer si se siguieran usando instrucciones de lectura a memoria con escalado:

Direccionamiento con escalado 8x: incrementar *%rax* de 1 en 1		Direccionamiento sin escalado, incrementar *%rax* de 8 en 8:
`movsd (%rdi,%rax,8), %xmm0`	→	`movsd (%rdi,%rax), %xmm0`
`addq  $1, %rax.`	→	`addq  $8, %rax`

El valor 8 corresponde con el tamaño de los elementos de los vectores **A**, **B** y **C**: sizeof(double) = 8 Bytes.

8. Calcula la **relación de mejora** (*speedup*) para el número total de instrucciones máquina ejecutadas.

El bucle del programa para ISA **x86-64** se compone de un único **bloque básico**, compuesto de 9 instrucciones, y el bucle no está desenrollado: hemos de considerar un promedio de 9 instrucciones / iteración. Comparando con la primera versión, que ejecutaba entre 12 y 13 instrucciones por iteración, esta nueva versión es siempre mejor.

Peor caso: speedup = 12 / 9 = 1,33 (33%) Mejor caso: speedup = 13 / 9 = 1,44 (44%)

El bucle del programa para ISA **rv64gc** se compone de tres **bloques básicos**, A, B y C, compuestos de 10, 1 y 1 instrucciones, respectivamente, y el bucle no está desenrollado. En el mejor caso, se ejecutan 10+1= 11 instrucciones / iteración, y en el peor caso se ejecutan 10+1+1= 12 instrucciones / iteración. Comparando con la primera versión, que ejecutaba entre 12 y 14 instrucciones por iteración, esta nueva versión es siempre igual o mejor.

Peor caso: speedup = 12 / 12 = 1,00 (igual) Mejor caso: speedup = 14 / 11 = 1,27 (27%)